Reporte de Monitoreo de Base de Datos

Segundo Avance — Proyecto Integrador

EQUIPO

Apaez Sotelo Alexis Jesus

Canchola Aguilar Alan Yahir

Jimenez Barcelata Isaac

Negrete Juárez Vanessa

Pérez Bosques Laura Lizet

Ramírez López Alicia Fernanda

Materia: Administración de Bases de Datos

Sistema: service-converter (Django + MySQL)

Fecha: 9 de abril de 2026

¿Cómo hicimos el monitoreo?

Para este reporte conectamos tres herramientas que trabajan juntas: **mysqld_exporter** se encarga de leer el estado interno de MySQL y exponer esa información como métricas, **Prometheus** recoge esas métricas cada 15 segundos y las guarda, y **Grafana** las muestra de forma visual en tiempo real.

La base de datos monitoreada es `converter_db`, alojada en el servidor `158.69.218.237:3350`. Durante la sesión generamos carga realizando inserciones, consultas y actualizaciones sobre la tabla `history_diagramhistory` para que las métricas reflejaran actividad real.

Herramientas utilizadas

Herramienta	Versión	Puerto	Función
mysqld_exporter	0.16.0	9104	Extrae métricas de MySQL y las expone en /metrics
Prometheus	3.5.1	9090	Recolecta y almacena las métricas en series de tiempo
Grafana	12.3.2	3000	Visualiza las métricas mediante paneles y dashboards

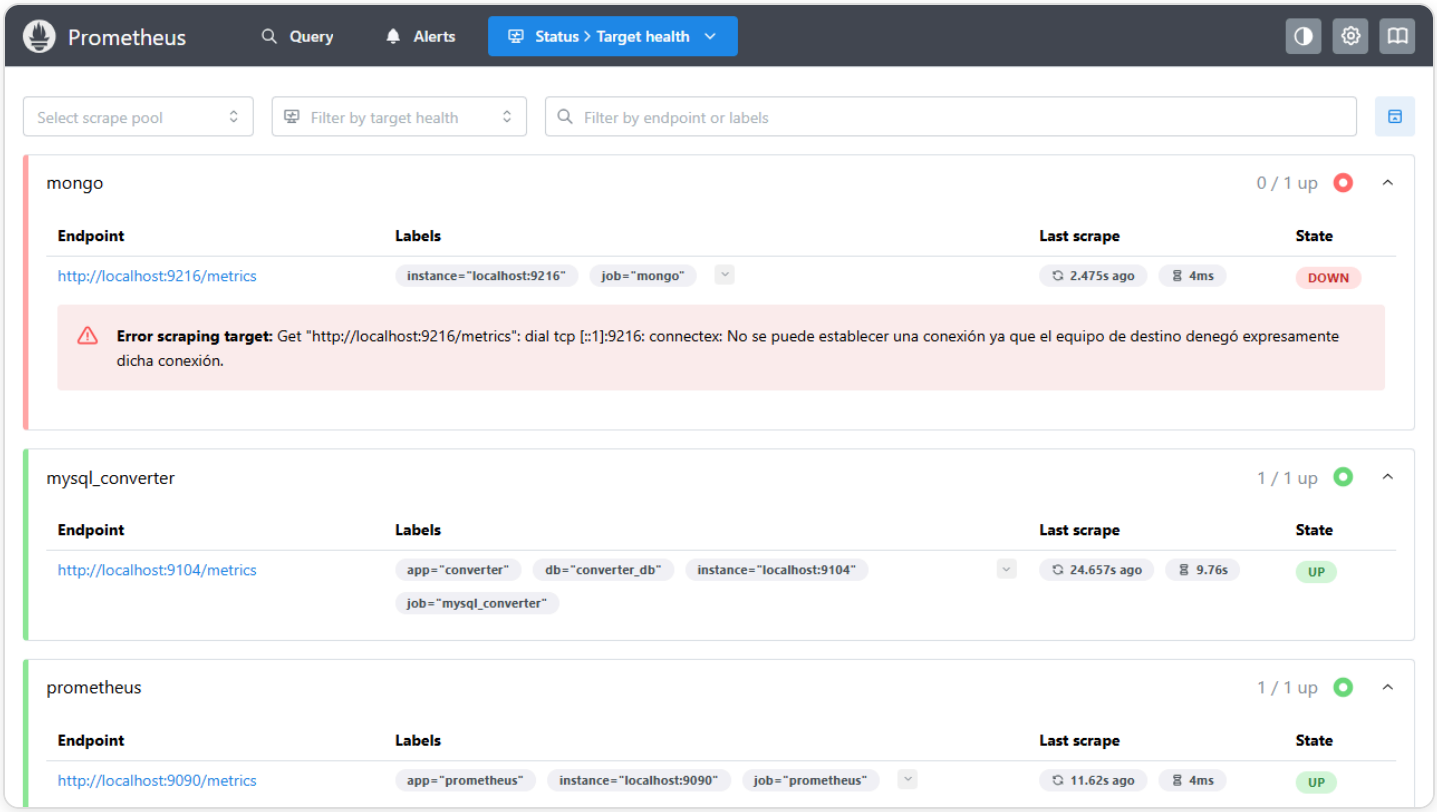

Figura 1 — Prometheus targets: mysql_converter en estado UP

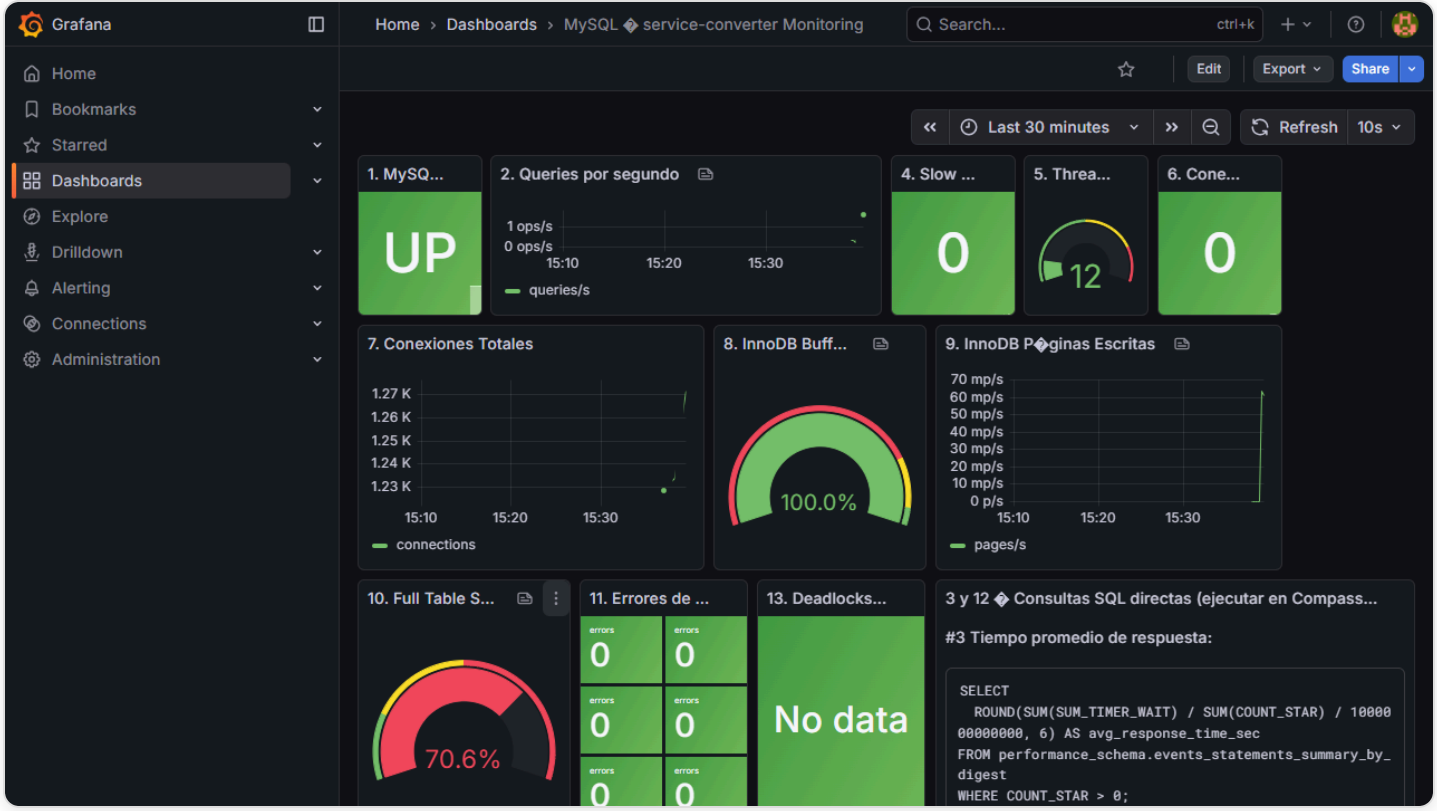

Figura 2 — Dashboard de monitoreo MySQL en Grafana

Métricas monitoreadas

1. Estado del servidor MySQL

UP — 1

```
mysql_up
```

Indica si el servidor MySQL está disponible. El valor **1** significa que está funcionando correctamente; **0** indicaría que está caído. Es la métrica más básica y fundamental: si esto falla, todo lo demás también falla.

2. Tasa de consultas por segundo

1.37 q/s

```
rate(mysql_global_status_queries[1m])
```

Mide cuántas consultas se ejecutan por segundo en el último minuto. Un valor de **1.37 consultas/seg** es bajo y saludable para desarrollo. En producción bajo carga alta puede llegar a cientos; si sube de golpe sin razón aparente puede indicar un problema.

3. Tiempo promedio de respuesta por consulta

N/A

```
SELECT ROUND(SUM(SUM_TIMER_WAIT)/SUM(COUNT_STAR)/1000000000000,6) AS avg_response_time_sec
FROM performance_schema.events_statements_summary_by_digest
WHERE COUNT_STAR > 0;
```

Esta consulta calcula cuánto tarda en promedio cada instrucción SQL. El usuario `converter_user` no tiene permisos sobre `performance_schema`, por lo que no fue posible obtener el valor. Para habilitarlo se necesitaría `GRANT SELECT ON performance_schema.* TO 'converter_user'@'%'`. Tiempos menores a 0.01 seg son excelentes; mayores a 1 seg indican cuellos de botella.

4. Consultas lentas acumuladas

0

```
mysql_global_status_slow_queries
```

Cuenta las consultas que tardaron más del umbral configurado en `long_query_time`. Que sea **0** es excelente: ninguna consulta ha tardado demasiado. Si empezara a subir sería señal de queries mal optimizados o índices faltantes.

5. Hilos conectados actualmente

30

```
mysql_global_status_threads_connected
```

Muestra cuántas conexiones activas hay abiertas con MySQL en este momento. **30 hilos** es moderado y normal. Este número se debe comparar con el límite `max_connections`; si se acerca al límite, nuevas conexiones serán rechazadas.

6. Intentos de conexión abortados

0

```
mysql_global_status_aborted_connects
```

Registra cuántas veces un cliente intentó conectarse pero falló antes de completar la autenticación. Un valor de **0** significa que no ha habido credenciales incorrectas, timeouts ni errores de configuración. Si creciera podría indicar un ataque o mala configuración en alguna app.

7. Conexiones totales desde el inicio

1,297

```
mysql_global_status_connections
```

Contador acumulado de todas las conexiones establecidas desde que el servidor arrancó. Con **1,297 conexiones** y el sistema relativamente reciente, indica un uso moderado y continuo. Lo importante es monitorear su tasa de crecimiento en el tiempo.

8. Eficiencia del buffer pool de InnoDB

100%

```
(1 - (rate(mysql_global_status_innodb_buffer_pool_reads[5m]) /
     rate(mysql_global_status_innodb_buffer_pool_read_requests[5m]))) * 100
```

Mide qué porcentaje de lecturas se sirven desde RAM en lugar del disco. Un **100%** significa que toda la data está cargada en memoria, resultando en lecturas ultrarrápidas. Valores bajo el 90% indicarían que el buffer pool es muy pequeño para los datos que se trabajan.

9. Páginas InnoDB escritas por segundo

0.0235 pág/s

```
rate(mysql_global_status_innodb_pages_written[5m])
```

Indica la velocidad a la que InnoDB escribe páginas de datos al disco. Un ritmo de **0.0235 páginas/seg** es muy bajo, normal para poca carga de escritura. Valores muy altos pueden indicar una carga intensiva que podría saturar el disco.

10. Porcentaje de full table scans

59%

```
(rate(mysql_global_status_select_scan[5m]) /
 rate(mysql_global_status_questions[5m])) * 100
```

Muestra qué porcentaje de los SELECT recorren tablas completas sin usar índices. Un **59%** es alto — idealmente debería estar bajo el 20%. Esto sugiere que varias consultas no aprovechan índices, lo que se vuelve costoso conforme las tablas crecen.

11. Errores totales de conexión

0

```
mysql_global_status_connection_errors_total
```

Acumula todos los errores que ocurrieron durante el proceso de conexión. El valor **0** confirma que la conectividad entre la aplicación y la base de datos ha sido perfecta durante todo el periodo monitoreado.

12. Fragmentación de tablas

Sin fragmentación

```
SELECT TABLE_SCHEMA, TABLE_NAME, DATA_FREE
FROM information_schema.tables
```

```
WHERE DATA_FREE > 0
ORDER BY DATA_FREE DESC;
```

Muestra qué tablas tienen espacio fragmentado producto de eliminaciones o actualizaciones frecuentes. El resultado vacío indica que **ninguna tabla tiene fragmentación**, lo cual es excelente. La fragmentación reduce el rendimiento y desperdicia espacio en disco.

13. Deadlocks en InnoDB

0

```
mysql_global_status_innodb_deadlocks
```

Cuenta los bloqueos mutuos entre transacciones. Un deadlock ocurre cuando dos operaciones se bloquean entre sí esperando recursos que la otra tiene reservados. El valor **0** indica que no ha habido ningún conflicto, normal en un sistema con baja concurrencia.

Resumen de resultados

#	Métrica	Valor obtenido	Interpretación
1	mysql_up	1 (UP)	Servidor en línea y funcionando
2	Queries por segundo	1.37 q/s	Carga baja, sistema saludable
3	Tiempo prom. respuesta	N/A	Sin permisos en performance_schema
4	Slow queries	0	Ninguna consulta lenta registrada
5	Threads conectados	30	Conexiones activas moderadas
6	Conexiones abortadas	0	Sin intentos fallidos de autenticación
7	Conexiones totales	1,297	Uso continuo desde inicio del servicio
8	Buffer pool hit rate	100%	Toda la data se lee desde RAM, óptimo
9	Páginas InnoDB escritas/s	0.0235	Carga de escritura muy baja
10	Full table scans	59%	Alto — requiere revisión de índices
11	Errores de conexión	0	Conectividad perfecta
12	Fragmentación de tablas	Sin fragmentación	Tablas en buen estado
13	Deadlocks InnoDB	0	Sin conflictos de transacciones

Acciones de mejora sugeridas

Revisar y agregar índices (prioridad alta)

El 59% de full table scans es la señal más preocupante del reporte. Habría que analizar qué columnas se usan en cláusulas WHERE, JOIN y ORDER BY y crear índices en ellas, especialmente en `history_diagramhistory.user_id` y `history_diagramhistory.created_at`, que son las más consultadas.

Otorgar acceso a performance_schema

Para medir el tiempo promedio de respuesta (métrica 3) se necesita ejecutar: `GRANT SELECT ON performance_schema.* TO 'converter_user'@'%';`. Esto permitiría identificar cuáles consultas específicas son las más lentas.

Configurar pool de conexiones en Django

Con 30 threads activos actualmente el sistema está bien, pero al crecer en usuarios se recomienda usar `CONN_MAX_AGE` en la configuración de Django o una herramienta como ProxySQL para reusar conexiones y evitar que se saturen.

Mantener el buffer pool hit rate por encima del 95%

Actualmente está en 100%, lo cual es óptimo. Si la base de datos crece en volumen, este porcentaje podría bajar. En ese caso se debería incrementar `innodb_buffer_pool_size` en la configuración de MySQL para mantener más datos

en memoria.